

REST API: Find the Speedster

Problem Description

Identify the fastest runner of a given **sex** in a specific **marathon**. A speedster is defined as the runner with the highest `top_speed`.

Access the marathon runner database via HTTP GET requests at:
`https://jsonmock.hackerrank.com/api/marathon`

Filtering and Pagination:

- To filter by sex, append `?sex=female` or `?sex=male`.
- The results are paginated. Access pages by adding `&page=num` to the query string.

Tie-breaking Rule:

If two runners have the same `top_speed`, return the one with **fewer stops taken** (`stops_taken`). If no runner matches the criteria, return an empty string.

Solution Logic

- **Initial Request:** Make a request to the API filtered by sex to get the total number of pages (`total_pages`).
- **Iteration:** Loop through every page from 1 to `total_pages`.
- **Data Processing:**
 - Filter runners by the provided marathon name.
 - Track the runner with the maximum `top_speed`.
 - Apply tie-breaking logic using `stops_taken` if speeds are equal.

Python Implementation

```
import requests

def findSpeedster(marathon, sex):
    url = f"https://jsonmock.hackerrank.com/api/marathon?sex={sex}"

    # Get the first page to find the total number of pages
    try:
        response = requests.get(url).json()
    except Exception:
        return ""

    total_pages = response.get('total_pages', 0)
    best_runner = None
    max_speed = -1.0
    min_stops = float('inf')

    # Iterate through all pages
    for page in range(1, total_pages + 1):
        page_url = f"{url}&page={page}"
        data = requests.get(page_url).json().get('data', [])

        for runner in data:
            # Check if the runner participated in the specific marathon
            if runner.get('marathon_name') == marathon:
                current_speed = float(runner.get('top_speed', 0))
                current_stops = int(runner.get('stops_taken', 0))

                # Update best runner based on speed and tie-breaker
                if current_speed > max_speed:
                    max_speed = current_speed
                    min_stops = current_stops
                    best_runner = runner.get('name')
                elif current_speed == max_speed:
                    if current_stops < min_stops:
                        min_stops = current_stops
                        best_runner = runner.get('name')

    return best_runner if best_runner else ""
```

Note: This solution assumes the `requests` library is available in the environment as indicated by the problem instructions.